

Model-Driven Software Development

Meta modeling



Miguel Campusano

SDU Software Engineering

Previous class

Motivation

Summary

Motivation

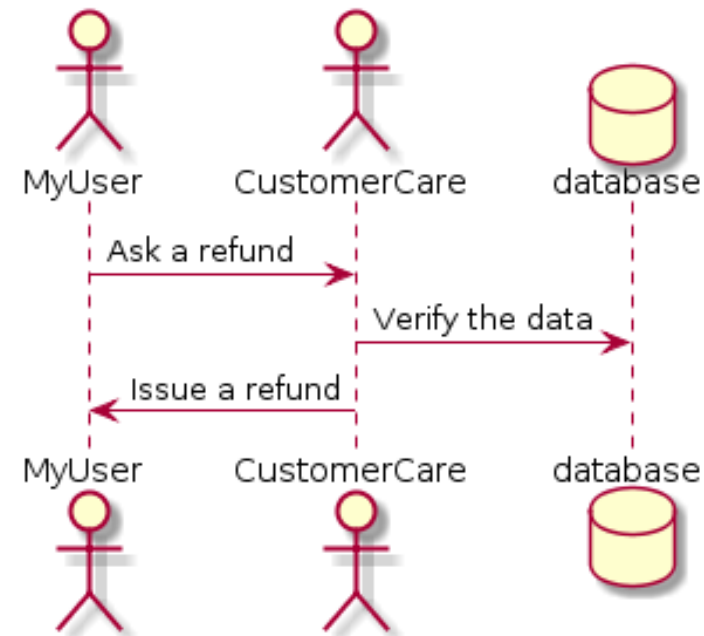
(Domain-Specific) Programming Languages (?)

Course Introduction



Before next class

Post a DSL example on itslearning



Motivation 1: Bugs: Solution

Automated code generation: correct by construction

- Develop correct code generator

- Automatically generate applications

- Profit! (well, not really)

Verification and Validation

- Manually implement software and formal verification of requirements

- Testing and more ...

Hypothesis: should be much more widely adopted!

Motivation 2: CvsP: Solution

MDSD: Model-Driven Software Development

Formal model

- UML, simulink, DSLs

- Syntax and semantics

- High-level and domain-specific

Relies on domain-specific notation

- Code generation or model interpretation

- Potentially huge gains in productivity (e.g., describe repetitive code once)

Verify safety, timing, security, ... , properties

Requires good tool support!

Motivation 3: Understanding Prog Lang

What is a Prog Lang? How do we define/implement them?

Compilers, interpreters, virtual machines are everywhere, how do they work?

What is a Turing machine, what is Turing completeness?

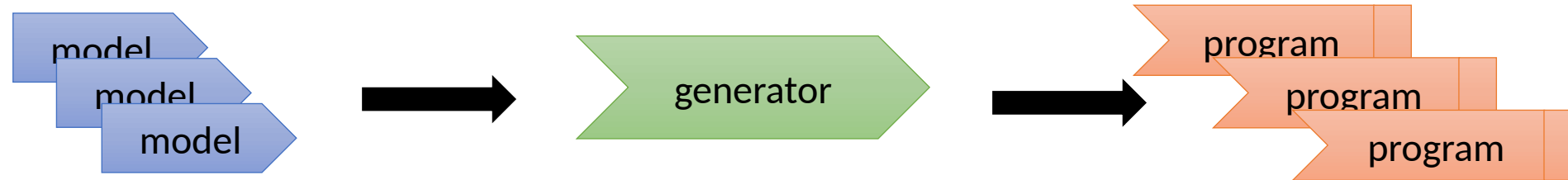
What are Regular Expressions (RE)? relation between regular expressions and state machines? Context-Free Grammars (CFG)? RE vs CFG?

How do type systems relate to verification and validation? What does soundness and completeness mean for type systems?

Visual programming languages? Low-code development?

...

Automated code generation



Technologies

Web tooling (Vue.js, Angular), modeling tools, DSLs

Compilers, generators, ...

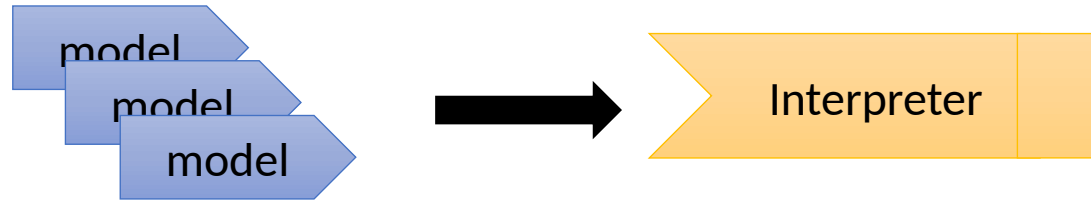
Alternatives

OO frameworks

Standard wizards found in IDEs

Outsourcing

Model-based execution



Technologies

JavaScript frameworks/API (jQuery), script engines, modeling tools, DSLs
Interpreters, virtual machines, ...

Alternatives

OO frameworks

Model-based tools that already *exist* (more or less)

Outsourcing

Domain-specific programming languages

Definition (almost)

- Computer programming language (BNF, ...)*

- Language nature (GCC linker scripts, ...)*

- Limited expressiveness (COBOL, ...)*

- Domain focus (always domain-specific)

Categories

- External DSLs: SQL, XML, CSS...

- Internal DSLs: hidden in fluent APIs (jQuery, LINQ, ...)

Domain-specific

Key Idea: Abstractions specific to domain

Problem domain / Family of problems

What is a relevant domain?

Interesting problems to solve

Solving problems is difficult (expressiveness, correctness)

Sensible abstractions (cohesion)

...

Discussion

Discussion 1: Meta model

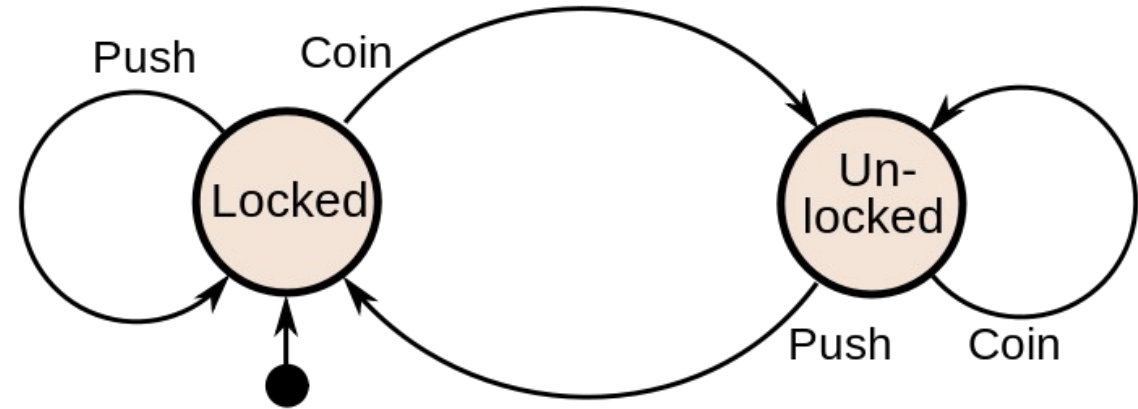
Metamodel

What is it?

Why is it important?

Metamodeling?

Have you done this before?



Discussion 2: Your Examples

Model-Driven Software Development

Meta modeling



Miguel Campusano

SDU Software Engineering

Today

Models and modeling

Our first example: class diagrams

Warm-up Internal DSL

Models

More than documentation

Model

Entity / Relation Diagram

Class diagram

Any UML implementation...

How do you normally use them?

Model

Entity / Relation Diagram

Class diagram

Any UML implementation...

How do you normally use them?

Documentation

Model implementation is *not formal*

Model as documentations: disadvantages

- Software systems are dynamic

 - Documentation needs to be maintained

- Models are the devs' interpretation

 - Devs still need to build programs manually

Model-driven system

Models are part of the development process

They are equal to code

We can *generate* code from models

Models are part of the software system

Domain is essential for model-driven system

Domain drives abstractions (models)

Model-driven system: advantages

Increase of productivity via automation

Quality and maintainability increase

Domain-experts can understand models

DSL Advantages

Improving development productivity

Communication with domain experts

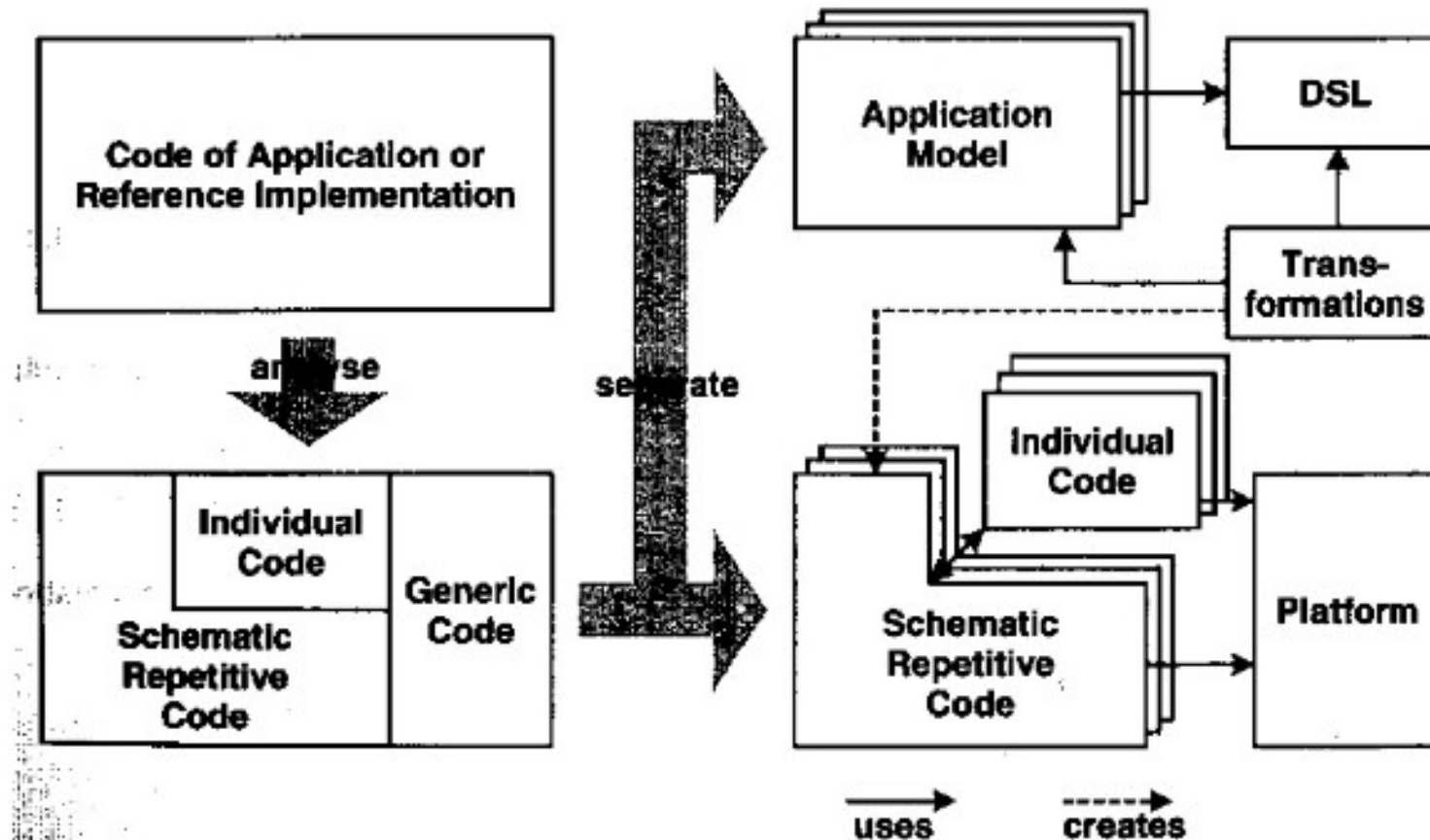
Change in execution context

Alternative computation model

Opportunities for verification, ...

Order of magnitude gain when done right!

Model-driven architectures



State Machines

Metamodel Introduction

State Machines

Computer Science

Mathematical model of computation

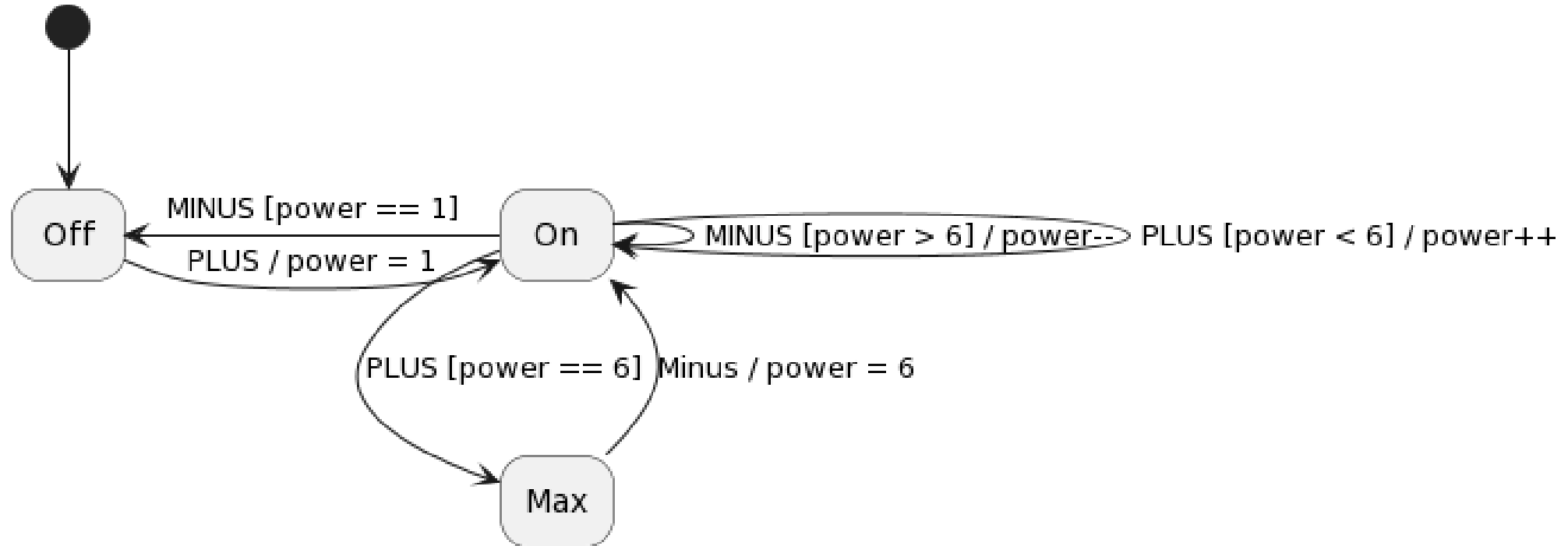
Robotic Engineer

Method for programming reactive systems (robotic behaviors)

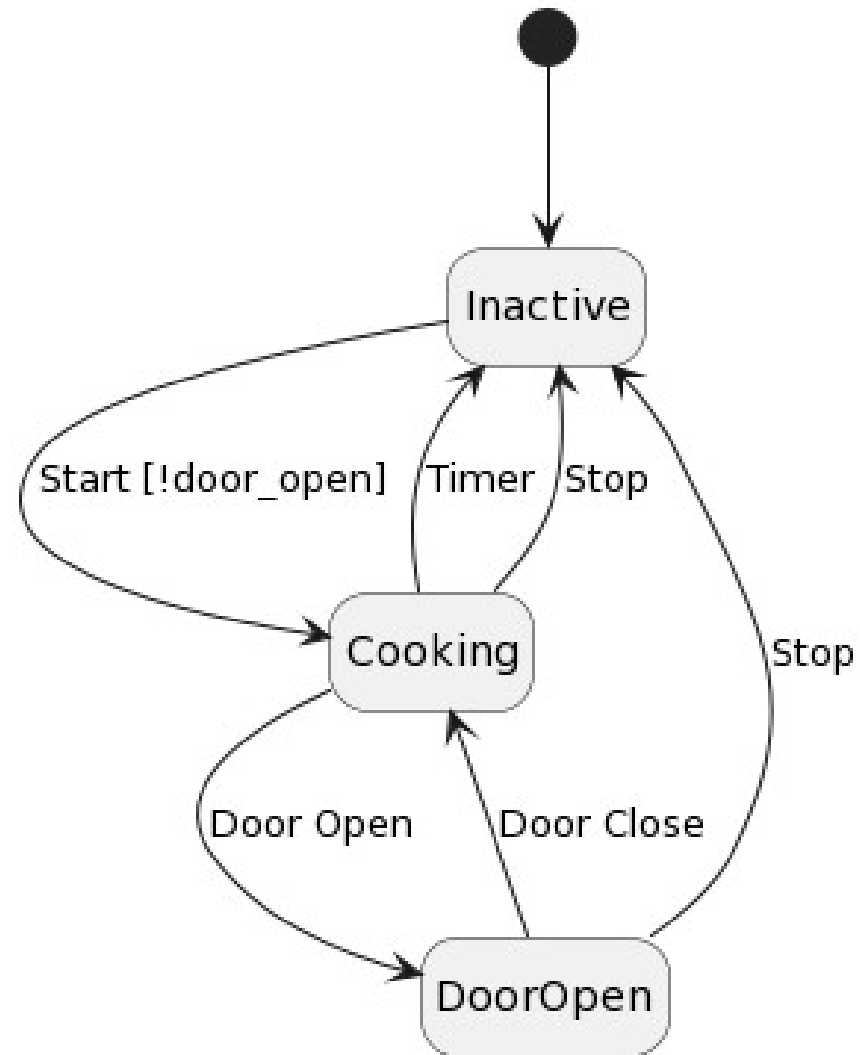
Software Engineer

DSL for programming event-based systems

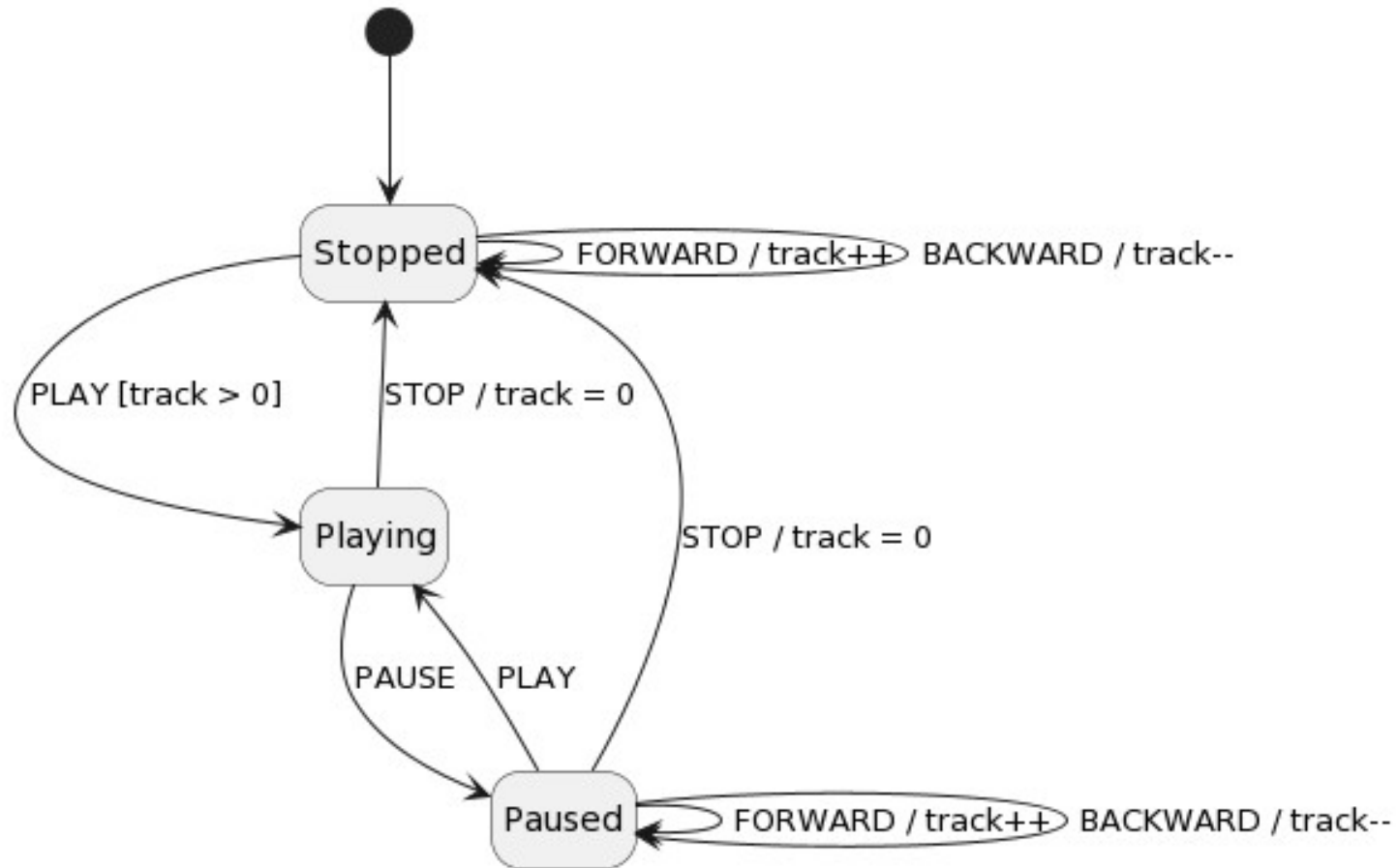
State Machines: Cooking hood



State Machines: Microwave Oven



State Machines: CD Player



State Machine metamodel

Rule of thumb

- 2 to 3 representative examples

- Abstract common properties

- Design (meta)model of examples

- Design DSL Syntax

Assignment 1

Internal DSL for state machines using a fluent API

Honest attempt: convince me that you worked

Normal: Java code

We have behavioral tests that you must pass

Write structural test yourself: check your metamodel design

Advanced: Any programming language + write tests + write pdf

Tests tell us that everything works

We cannot run the code written in other languages

You need to convince us that your solution works

Assignment 1

Internal DS

Honest att

Normal: Java

We have k

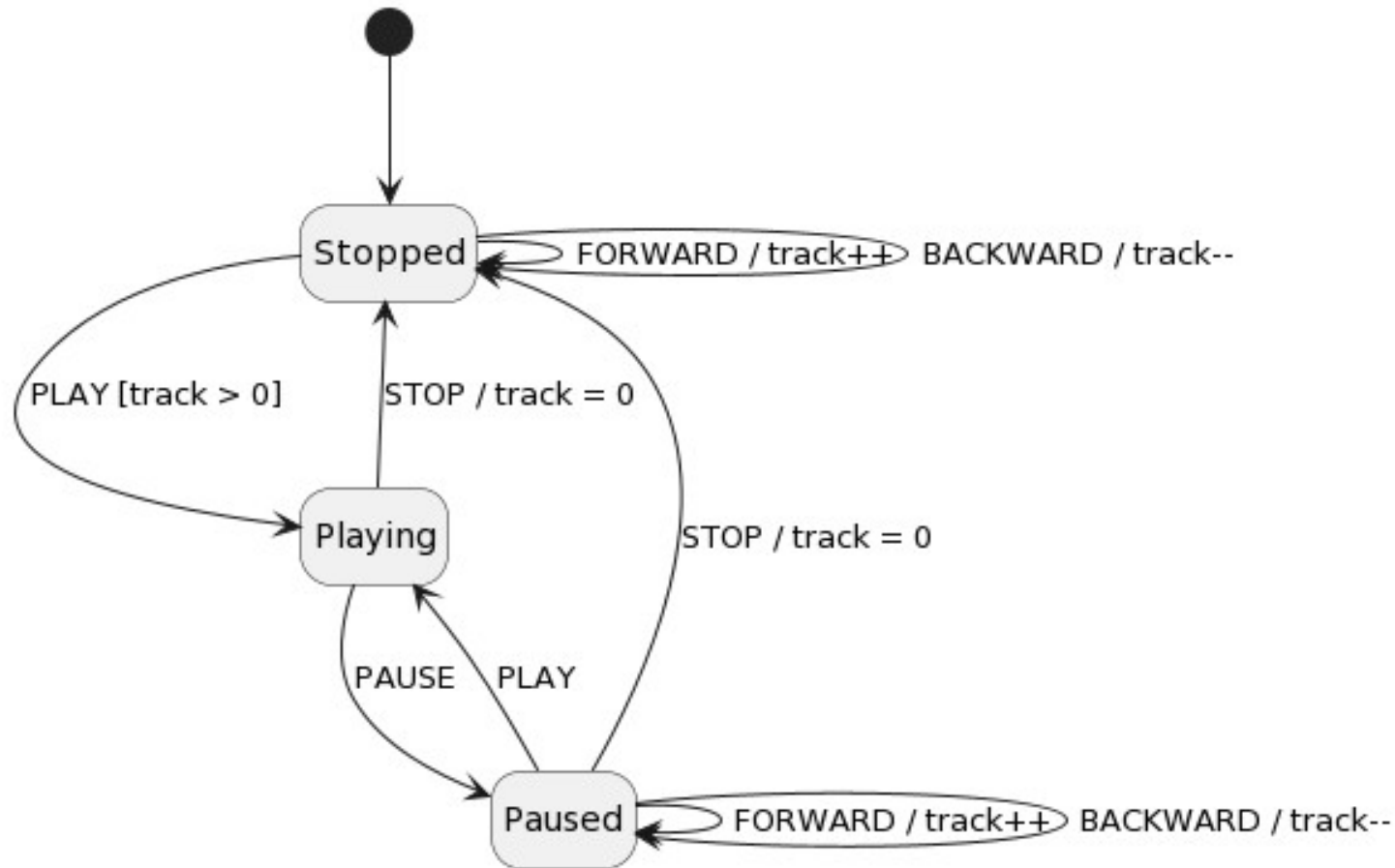
Write stru

Advanced: A

Tests tell u

We cannot

You need



Metamodeling

An exercise

Metamodeling

Class Diagram

Rule of thumb

- 2 to 3 representative examples

- Abstract common properties

- Design (meta)model of examples

- Design DSL syntax

Internal DSL

An introduction

Internal DSLs

DSL embedded into the syntax and semantics of a host language

Advantages: simple implementation, inherits advantages of host language

Disadvantages: syntax limitation, inherits disadvantages of host language

Host-language dependent

OOP languages (Java, C#, Python): OOP *fluent interface*

Developers culture (Ruby, Smalltalk, etc): *business as usual*

C : Macros

...

Internal DSLs: techniques

Abstraction through metamodel

Interpretation more than compilation

Convenient syntax through patterns and *cleverness*

- Method builder / factory

- Method chaining

- Macros

- ...

Internal DSLs: techniques

Abstraction through metamodel

Interpretation more than compilation

Convenient syntax through patterns and *cleverness*

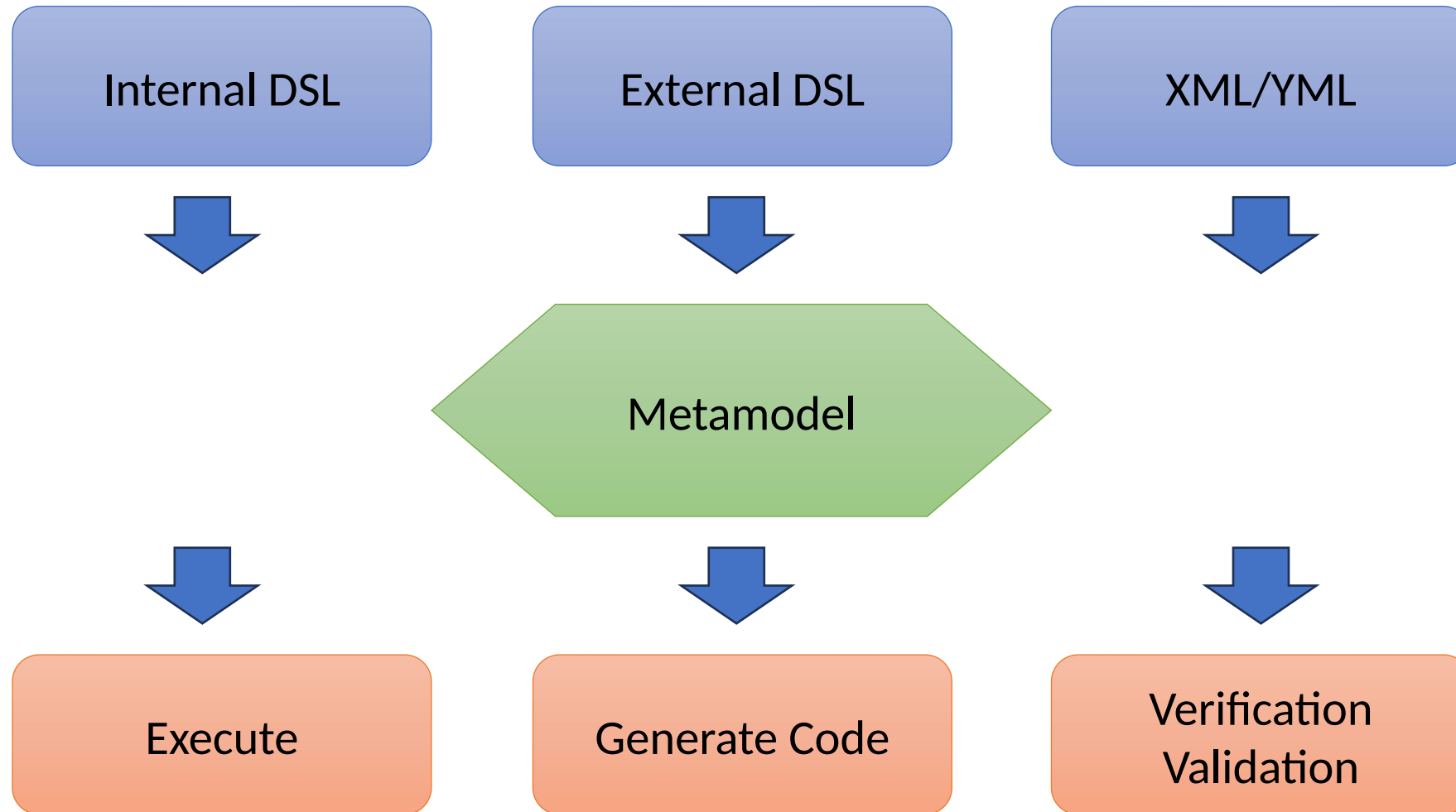
- Method builder / factory

- Method chaining

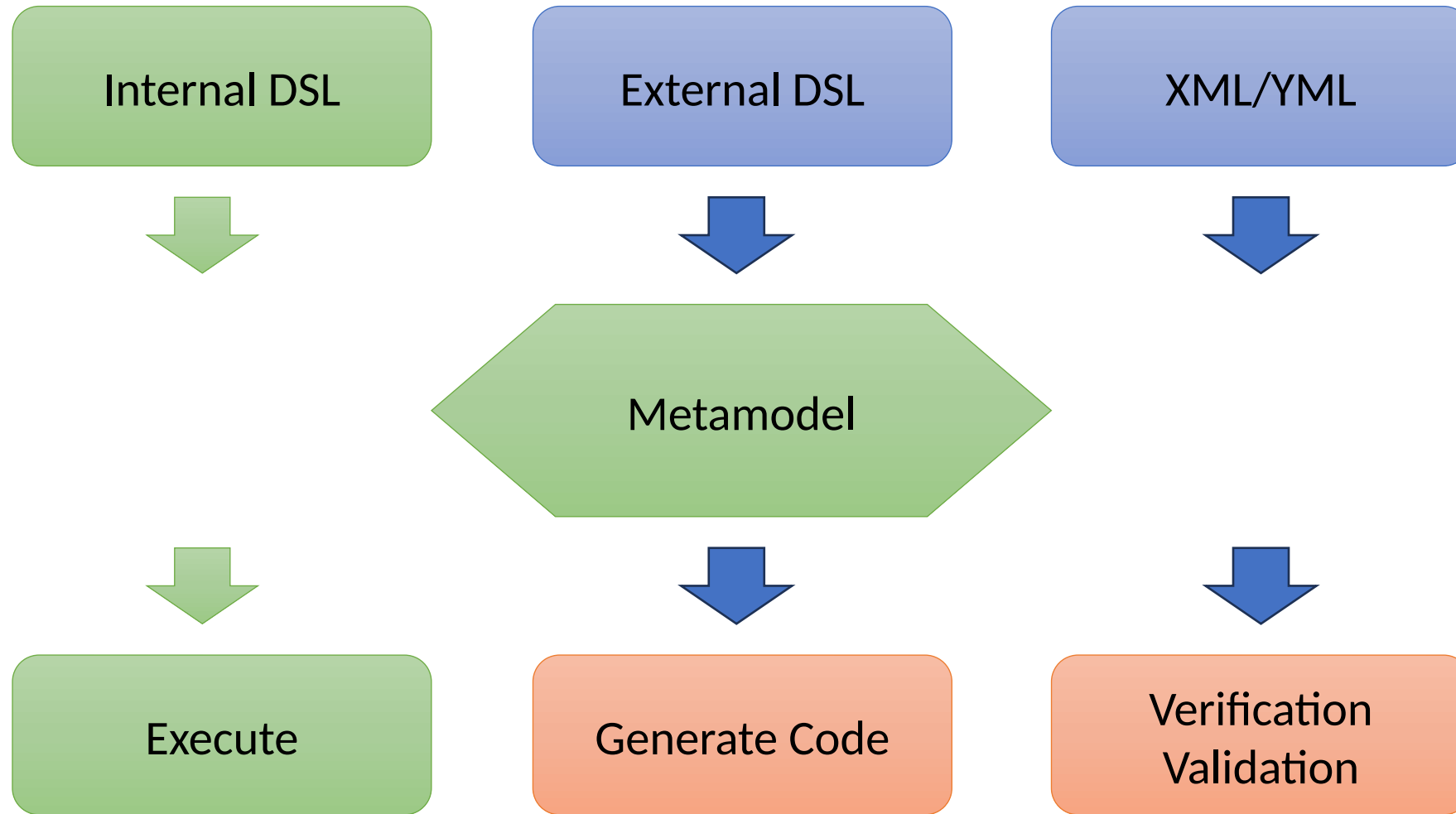
- Macros

- ...

Metamodel



Metamodel: Internal DSL interpretation



Internal DSL: GUI example



```
@Override
public void build() {
    frame("Counter").
        label("display", "0").
        newline().
        button("Less").button("More").
        newline().
        button("Reset").button("Quit").
        newline();
}

@Override
public void actionPerformed(ActionEvent event) {
    String button = event.getActionCommand();
    if("Less".equals(button)) setDisplay(Integer.toString(--value));
    else if("More".equals(button)) setDisplay(Integer.toString(++value));
    else if("Reset".equals(button)) setDisplay(Integer.toString(value=0));
    else if("Quit".equals(button)) System.exit(0);
    else System.err.println("Warning: unknown event "+event);
}
```


Internal vs External DSL

First approach to external DSL

Internal DSL

Host language generates a model

Advantages of internal DSLs

- Quick to implement, natural evolution

- No additional tools, languages, infrastructure

Disadvantages of internal DSLs

- Model-based API can be cumbersome and confusing

- Lack of static checking, IDE support

- Mix of host language and DSL enables abuse

External DSL

- Compiler that translates text into a model

- Feels more like a *real language*

 - Dedicated grammar (IDE, language, syntax, ...)

 - Natural fit for code generation

- Requires construction of a *real language*

 - Significant work to build compiler infrastructure

 - Even more work to provide proper IDE support

 - Complex problem domain

Project

Group project plan

Week 1: Choose project

Week 2: (Meta)modeling

- Define 2 to 3 representative examples

- Abstract common properties

- Design (meta)model of examples

- Design DSL syntax

Summary

Summary

Models and modeling

Our first example: class diagrams

Warm-up Internal DSL

```
@Override
protected void build() {
    entity("Person").
        attribute("name", "String").
        attribute("age", "Number").
    entity("Student").
        attribute("id", "Number").
    entity("Teacher").
    entity("Course").
        attribute("title", "String").
    inheritance("Person", "Student").
    relation("Inheritance", "Person", "Teacher").
    relation("1-1", "Course", "Teacher").
    relation("Many-Many", "Course", "Student") ;
}
```

